Near Data Processing opportunities for context engineering

Jon Hermes

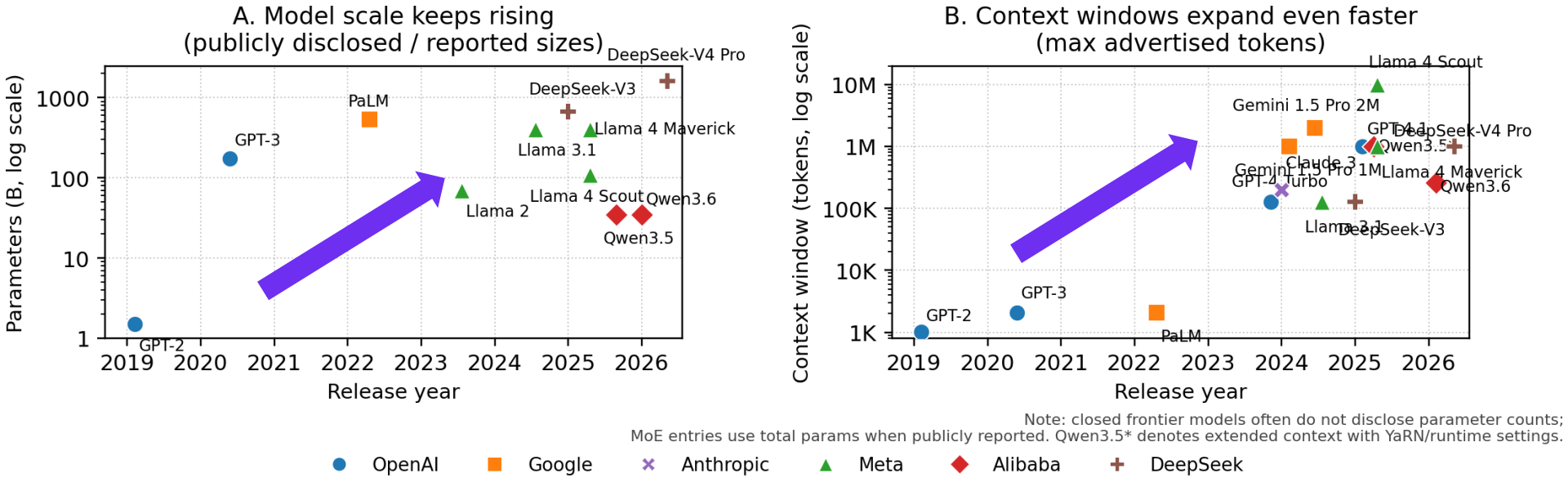
Arm



OCP FUTURE TECHNOLOGIES SYMPOSIUM

October 14, 2026
San Jose, California

Headline: "Contexts are growing"



Bigger Models

more parameters and embedded knowledge

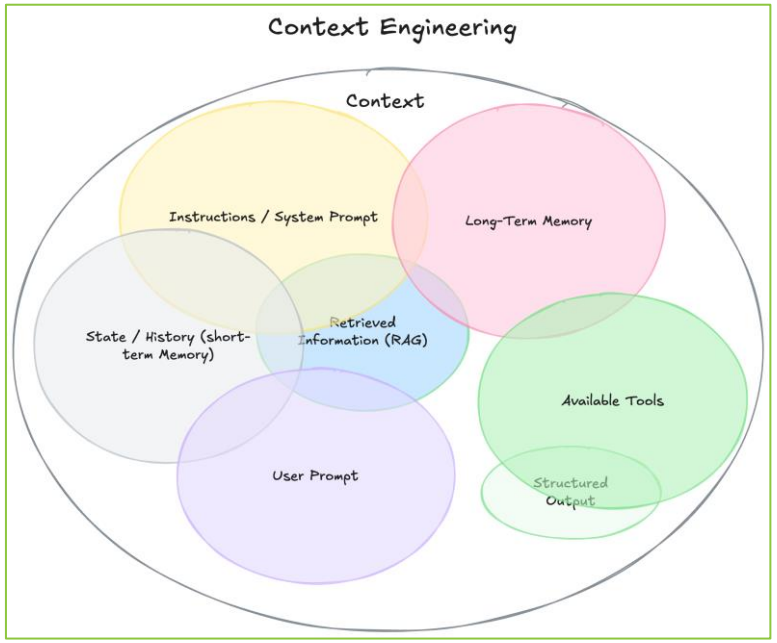
Longer Context

more tokens available at inference

Newer, bigger AI models using longer contexts to support new capabilities:

- Instruction following
- Tool use, function calling
- Reasoning and Planning

But what is context?



Token-level context

Token-level context is the most “context”-like form. The context exists as explicit language or structured text that can be inserted back into the prompt.

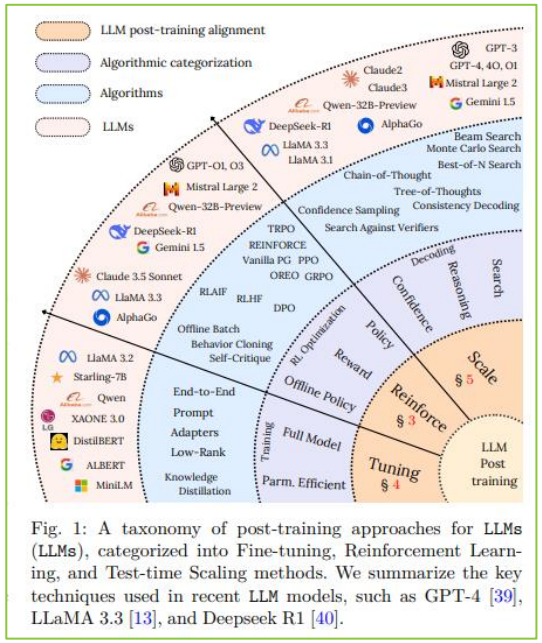
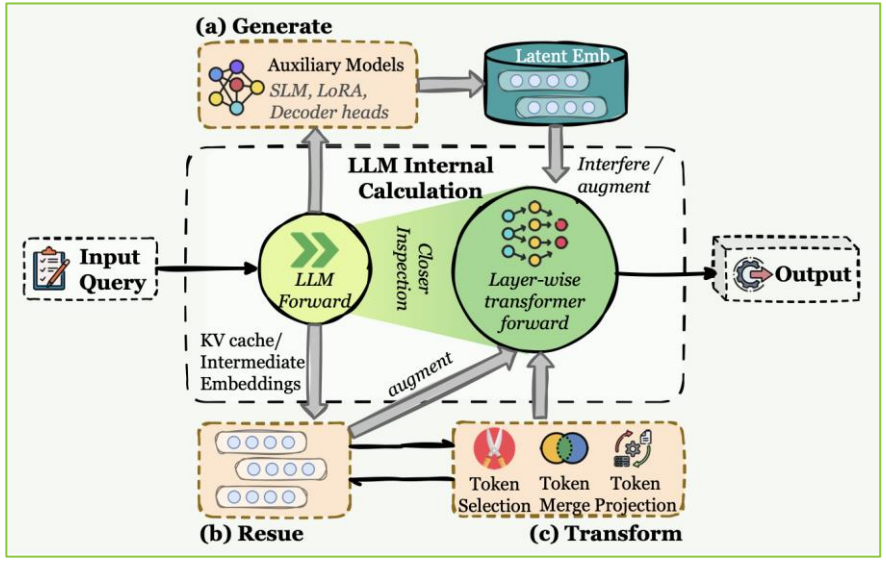


Fig. 1: A taxonomy of post-training approaches for LLMs (LLMs), categorized into Fine-tuning, Reinforcement Learning, and Test-time Scaling methods. We summarize the key techniques used in recent LLM models, such as GPT-4 [39], LLaMA 3.3 [13], and Deepseek R1 [40].

Parametric context

Parametric context means the information is encoded in learned parameters, such as Fine-tuning, LoRA/adapters, experience distillation, RL-updated policies



Latent context

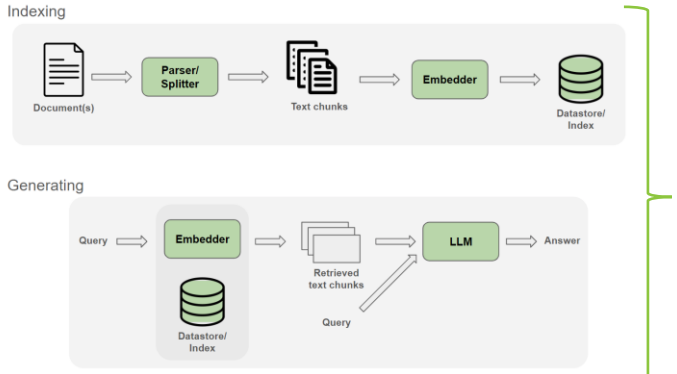
Latent context stores information in compressed continuous representations: hidden states, embeddings, KV caches, learned memory slots, or other non-text vectors.

3 types of context forms

Token-level memory, Parametric memory, and Latent memory [1]

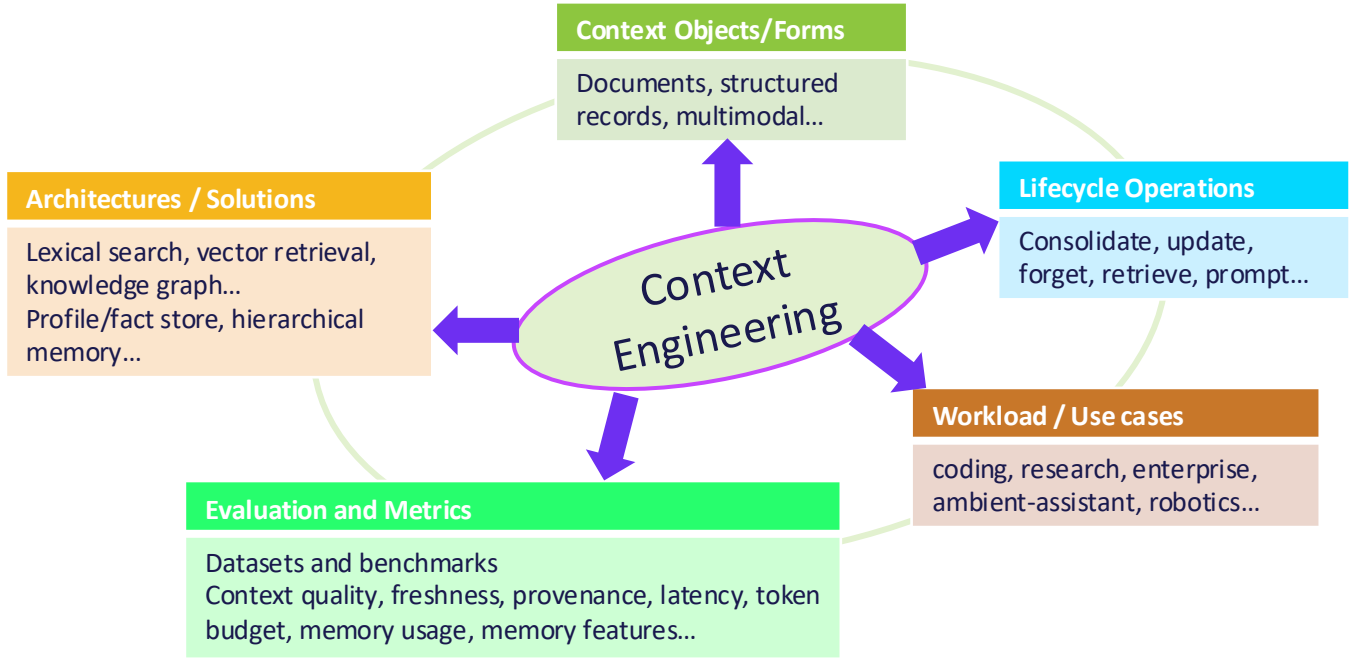
[1]. Memory in the Age of AI Agents: A Survey, https://arxiv.org/pdf/2512.13564

And what is context engineering?



Retrieval Augmented Generation

is just one part of context engineering



"Context Engineering" is a system design

It provides an externalized and evolving cognitive layer that can remain coherent, personalized, adaptive, and effective across **long-horizon** tasks. This is increasingly important in the burgeoning Agentic AI era which requires longer, often multi-turn contexts.

Sketch of LLM inference

Traditional P/D Disaggregation

The expectation for inference is that:

- users or agents make queries,
- which run through a compute-intensive Prefill phase (primarily on GPUs) to generate KV,
- which are then used in a memory-intensive Decode phase to generate full responses.

This view of inference assigns no time to context work, and the Orchestrator in this diagram only exists for coordination between the two halves.

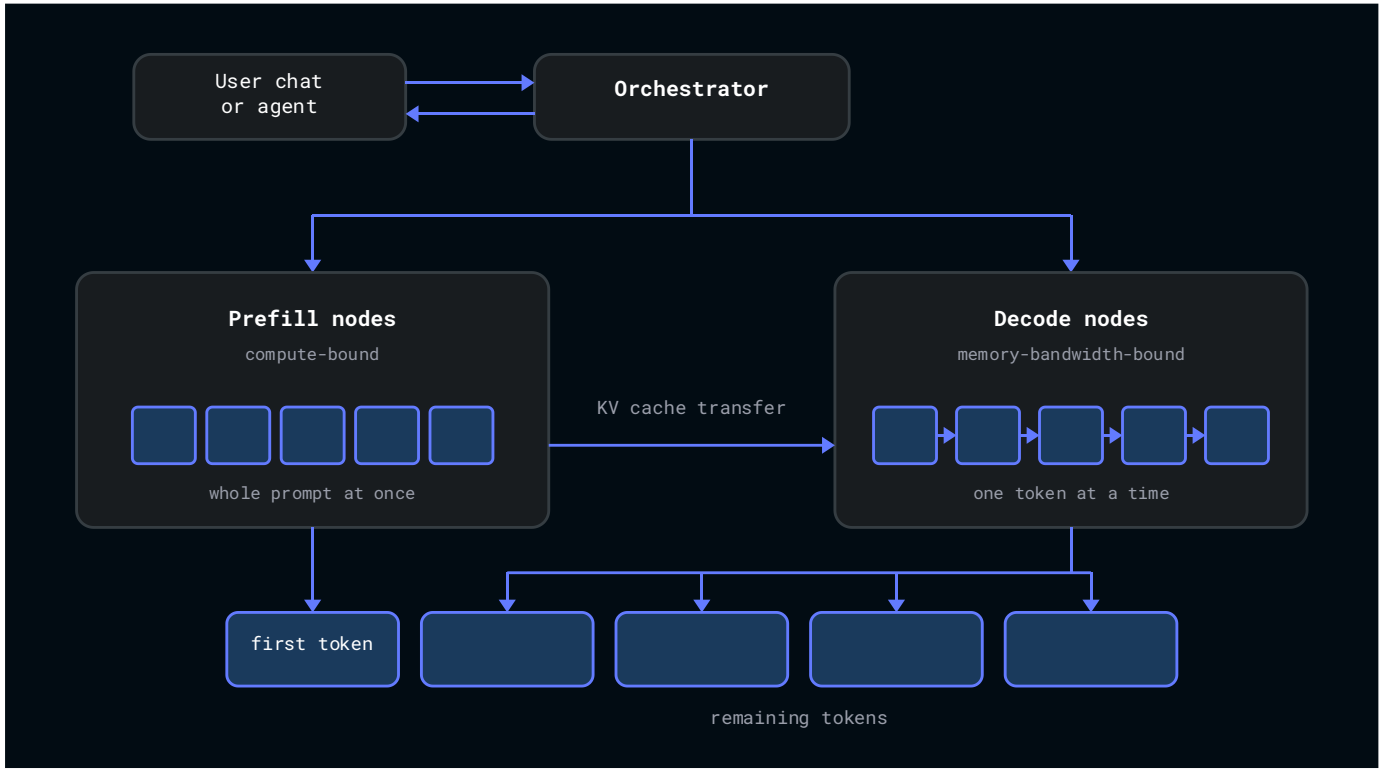


Image from Modular: https://handbook.modular.com/inference-optimization/prefill-decode-disaggregation/

Updated Sketch

A third pillar of computation for inference

We instead see inference as three primary parts:

- First, a pre-prefill "Context" section,
- Then Prefill,
- Then Decode.

Arithmetic Intensity and bounds

Prefill is known to be high arithmetic intensity and thus compute-bound, and decode is GEMV-based and thus known to be memory-bound. How is context work bounded?

In our experiments, we've found context work to be primarily **memory-bound**, which creates a sandwich of dense compute in between two sections of sparse compute with mem- and io-waiting.

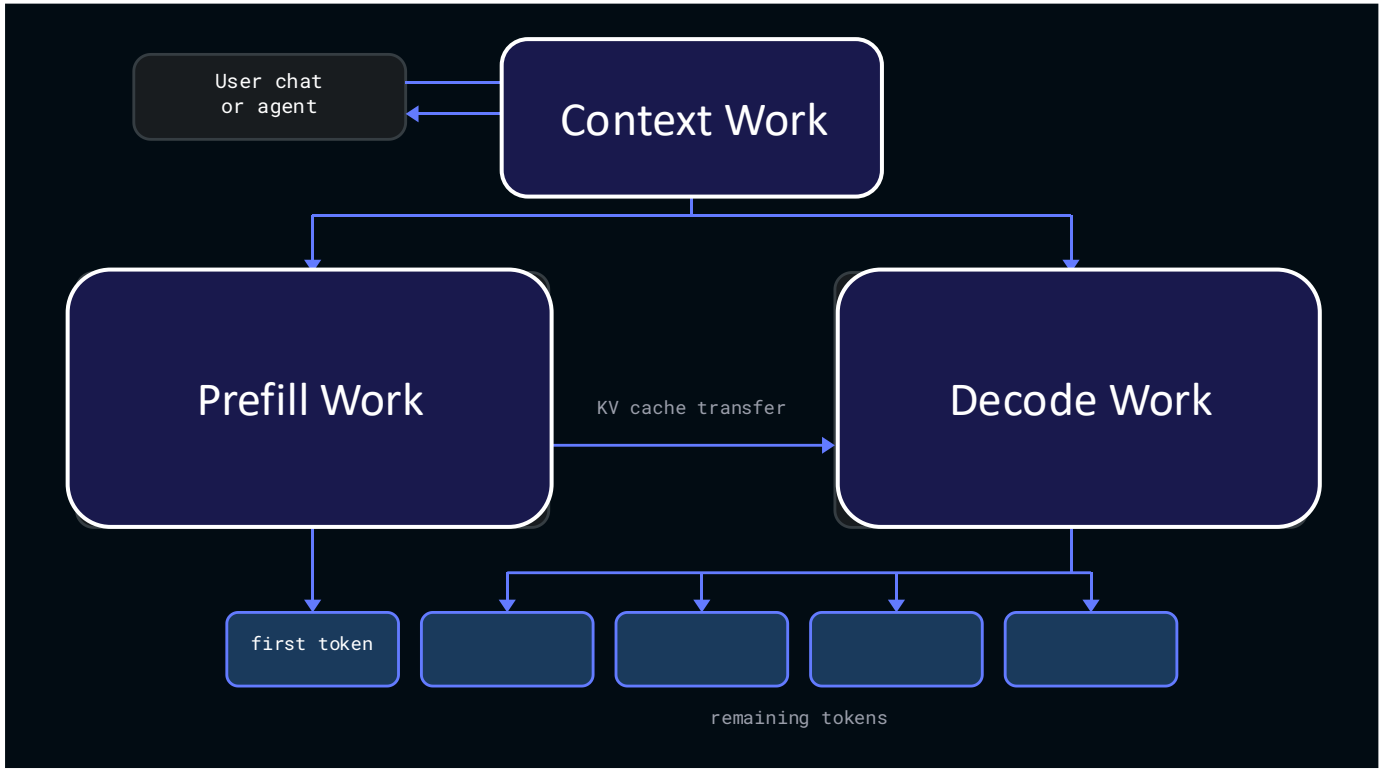


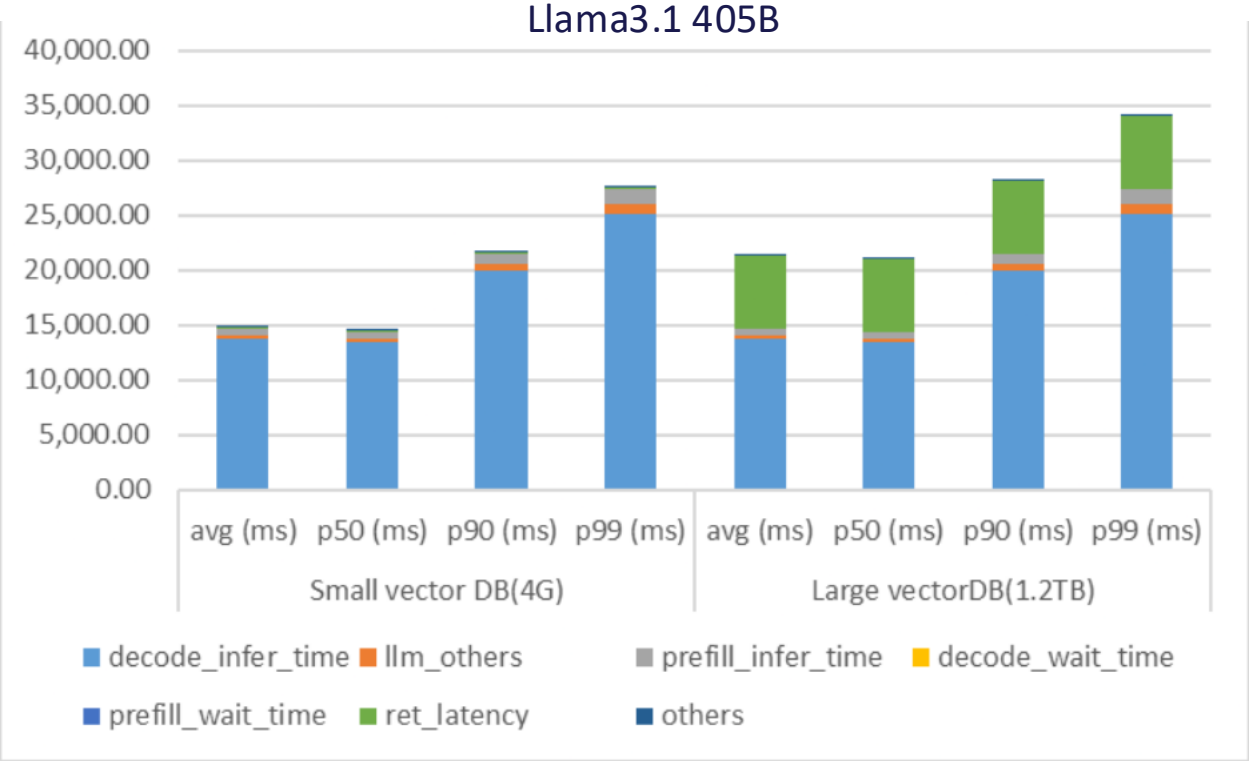
Image from Modular: https://handbook.modular.com/inference-optimization/prefill-decode-disaggregation/

Context work is significant

Profiling recent models has shown that just RAG within context work is a large amount of total time spent, approaching **30%** in some cases.

Decode still dominates total time, but as context length and context density grows we can see them taking equal time, or potentially overtaking.

Prefill usually takes up a noticeable amount of total time, though its cost is heavily attenuated by proper prefill caching.

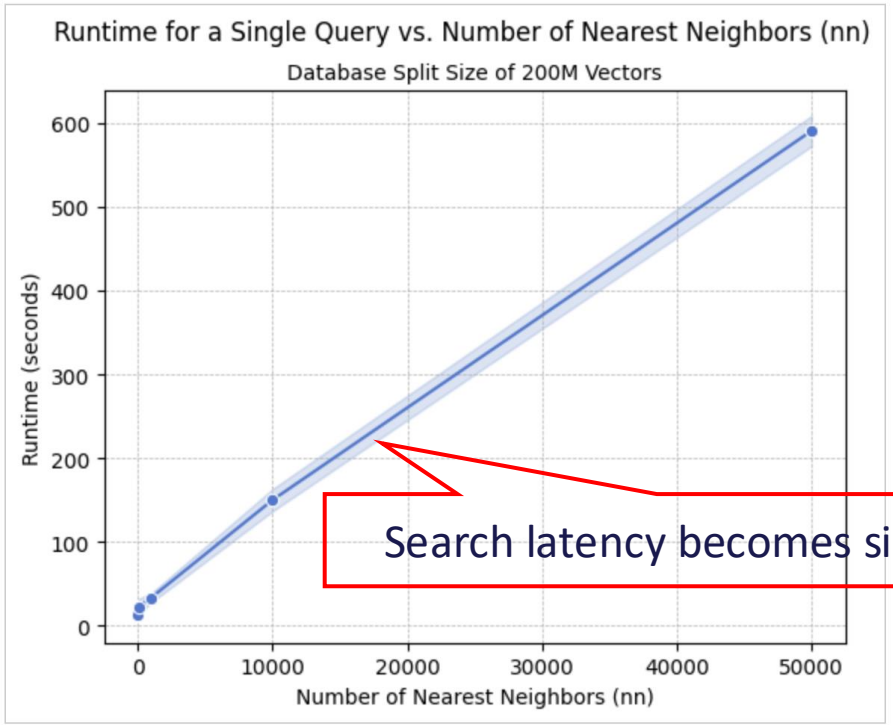


Relative time spent in tasks

RAG profiling with varying vectorDB sizes

Context work is significant (cont.)

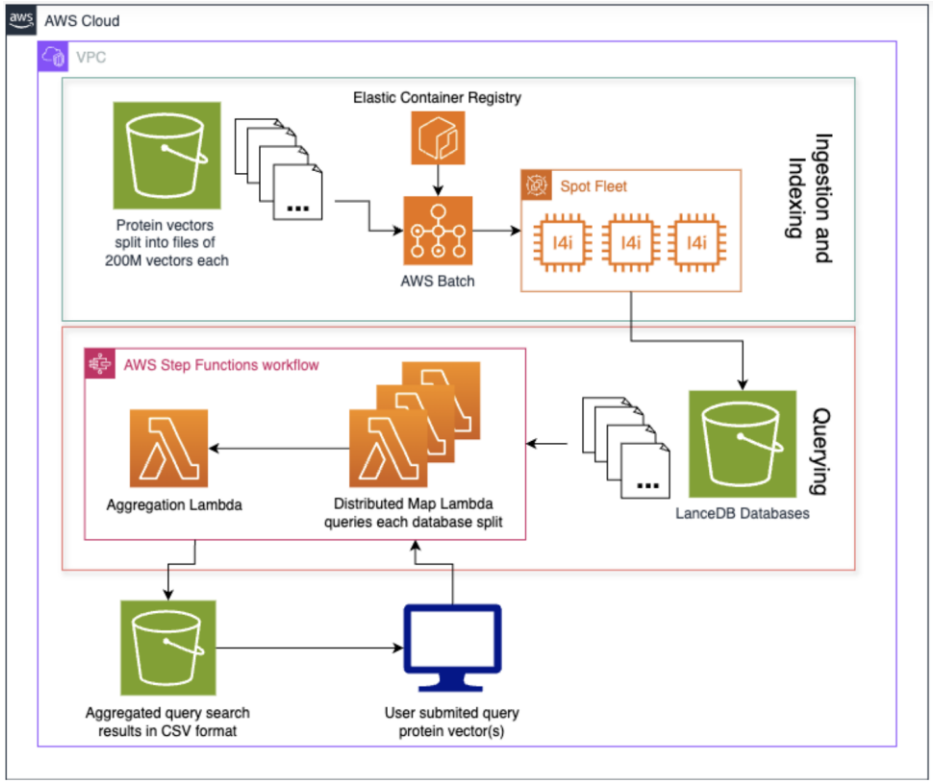
An example database created by Metagenomi consisted of 3.5 billion vector embeddings produced by AMPLIFY, of dimension 960. Ingesting and indexing these 3.5B vector embeddings in split sizes of 200M vectors on i4i.8xlarge instances took 108 total compute hours. Because this solution is serverless and can be queried directly from its S3 object store, the only fixed cost of this database is its storage footprint on Amazon S3 (for an indexed database of 3.5B vectors, this is approximately 12.9 TB).



AWS Architecture Blog

A scalable, elastic database and search solution for 1B+ vectors built on LanceDB and Amazon S3

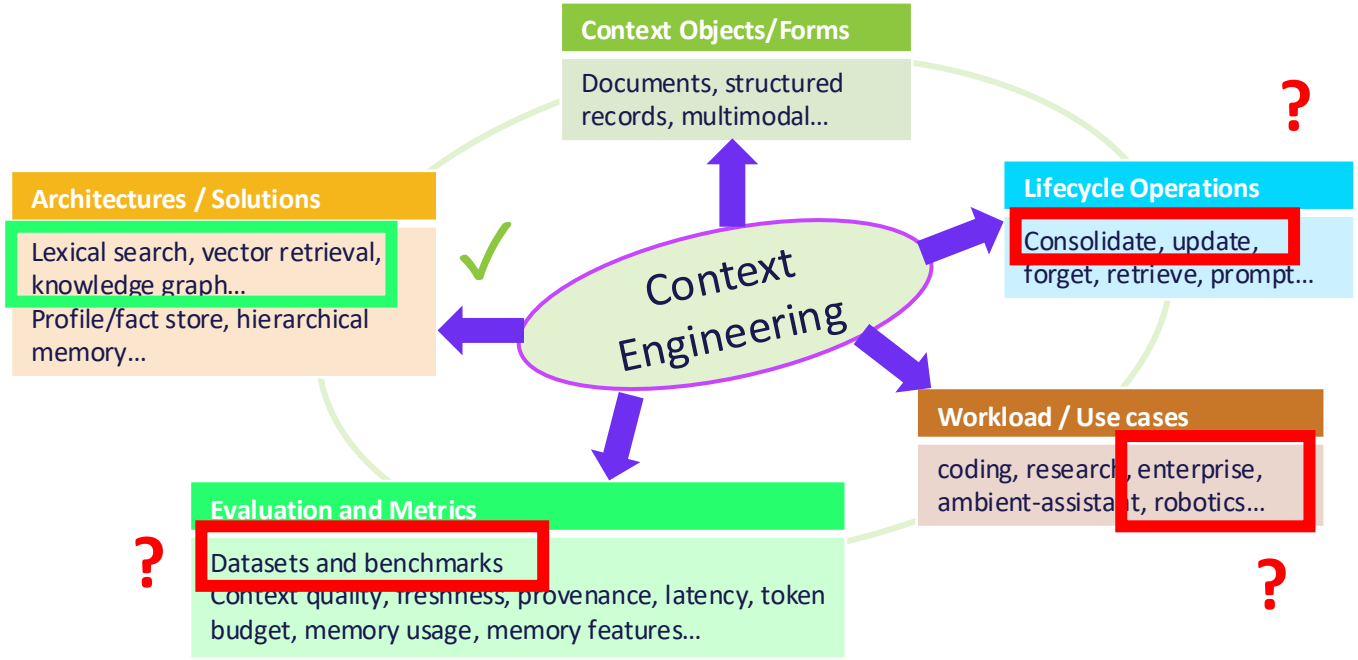
by Audra Devoto, Christopher Brown, Patrick O'Connor, Owen Janson, and Pavel Novichkov | on 22 SEP 2025 | in Advanced (300), Amazon Simple Storage Service (S3), AWS Batch, AWS Lambda, AWS Step Functions, Customer Solutions, Innovation and Reinvention, Life Sciences, Serverless, Startup, Technical How-to | Permalink | Share



https://aws.amazon.com/blogs/architecture/a-scalable-elastic-database-and-search-solution-for-1b-vectors-built-on-lancedb-and-amazon-s3/

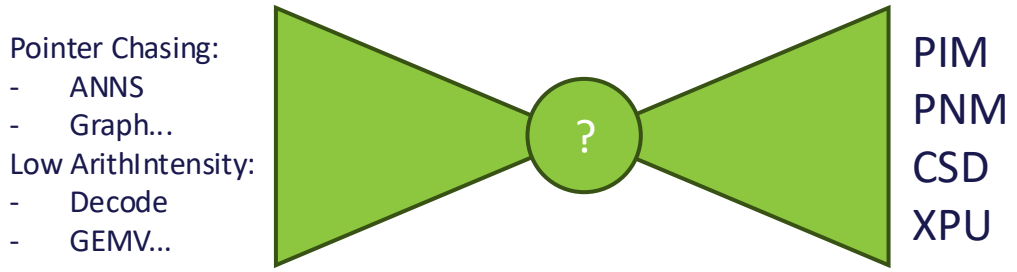
Context work and NDP

- Many research papers and blogs present the opportunity for Near Data Processing to accelerate Approximate Nearest Neighbor Search (ANNS), a key function for vectorDB, for RAG:
  - CXL-ANNS (ASPLOS'23), NDSEARCH (ISCA'24, Samsung), UDON (HCDS'24, Arm), COSMOS (IEEE CAL'25, Samsung), IKS (ASPLOS'25), Bauhaus (IEEE TC'26, SK hynix), SpANNS (ArXiv'26), FaTRQ (ArXiv'26)
  - Marvell's blog: "5 Times More Queries per Second: What CXL Compute Accelerators Can Do for AI" (Dec 25)
  - Many more...
- Do we have other opportunities for context engineering or agentic harness Near Data?
  - Anywhere that we see this trifecta is ripe for acceleration: **Large data volume, local irregular accesses, and limited compute/low arithmetic intensity.**



NDP's Bowtie Problem

- The primary obstacle to adoption of NDP accelerator devices is not necessarily finding the good use cases for offload, it's **software support** to be able to build and test these new forms of acceleration.
- The shape of the problem is a bowtie:
  - Large LHS with the various different software and functions that are good to offload
  - A centerpiece which transforms the LHS into a form the RHS can accept
  - Large RHS with the various different NDP devices (PIM, PNM, CSD, cHBM, DPU/TPU/XPU)



- A pure vertical solution can provide *some* of the beneficial offload on *specific* hardware, but the ideal software solution should allow for any arbitrary offload onto whichever hardware is available.

Conclusion

Context work is a significant portion of total inference time, and appears to be trending toward taking an even larger portion of total time.

Context work on the whole is prime for offloading onto NDP as it is often memory-bound and low arithmetic intensity.

We need a solution to the bowtie problem, to make it much easier to pick and test offload targets, and to design and test ever better NDP accelerator hardware.

OCP FUTURE TECHNOLOGIES SYMPOSIUM

October 14, 2026
San Jose, California

